南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Advanced Programming

Lab 7, shared library, function and memory

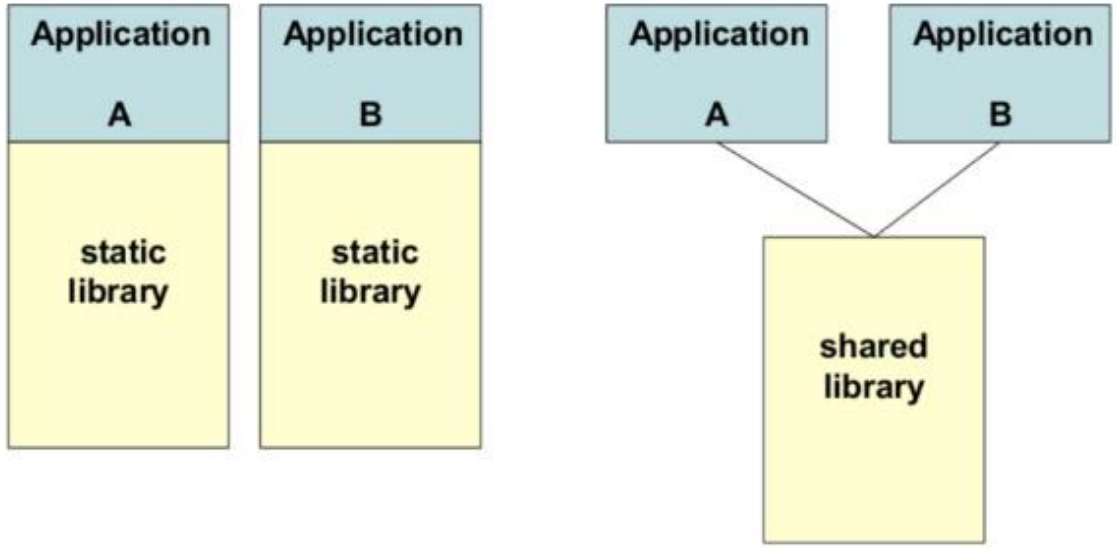
廖琪梅，王大兴，于仕琪，王薇

BY NC SA

Topic

- Shared library
  - Build
  - Use
  - Makefile
  - CMakelists.txt
- Function and Memory
  - function address
  - maps, executable memory
- Exercises

BY NC SA



	advantages	disadvantages
Static Library	1. Make the executable has fewer dependencies, has been packaged into the executable file. 2. The link is completed in the compilation stage, and the code is loaded quickly during execution.	1. Make the executable file larger. 2. Being a library dependent on another library will result in redundant copies because it must be packaged with the target file. 3. Upgrade is not convenient and easy. The entire executable needs to be replaced and recompiled.
Dynamic Library	1.Dynamic library can achieve resource sharing between processes, there can be only one library file. 2. The upgrade procedure is simple, do not need to recompile.	1. Loading during runtime will slow down the execution speed of code. 2. Add program dependencies that must be accompanied by an executable file.

BY NC SA

Building a shared library

- Suppose we have written the following code:

```
// mymath.h
#ifndef __MY_MATH_H__
#define __MY_MATH_H__
float arraySum(const float *array, size_t size);
#endif
```

```
// mymath.cpp
#include <iostream>
#include "mymath.h"

float arraySum(const float *array, size_t size)
{
   if(array == NULL)
   {
      std::cerr << "NULL pointer!" << std::endl;
      return 0.0f;
   }
   float sum = 0.0f;
   for(size_t i = 0; i < size; i++)
      sum += array[i];
   return sum;
}
```

```
// main.cpp
#include <iostream>
#include "mymath.h"
int main()
{
   float arr1[8]{1.f, 2.f, 3.f, 4.f, 5.f, 6.f, 7.f, 8.f};
   float * arr2 = NULL;

   float sum1 = arraySum(arr1, 8);
   float sum2 = arraySum(arr2, 8);

   std::cout << "The result1 is " << sum1 <<
std::endl;
   std::cout << "The result2 is " << sum2 <<
std::endl;

   return 0;
}
```

BY NC SA

Building shared libraries

- A **shared library** packs compiled code of functionality that the developer wants to **share** with other developers.
- Shared libraries in linux are **.so** files.
- Remember to use arguments "**-shared**" and "**-fPIC**" when building it.
- Now we should see "libmymath.so" in the directory

The name of .so must be started with "**lib**" followed by the .cpp name in which a function is defined.

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ g++ -shared -fPIC -o libmymath.so mymath.cpp
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ ls
libmymath.so  main.cpp  mymath.cpp  mymath.h
```

Create a shared library.

Generate Position-Independent-Code.

CC BY NC SA

Using shared library

- Now we can use the ".so" shared library.
- Let's compile "main":

"lmymath" indicates to use "libmymath.so"

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ g++ -o main main.cpp -L. -lmymath
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ ls
libmymath.so  main  main.cpp  mymath.cpp  mymath.h
```

"-L." indicates to find a library file in the current directory.

- **-L**: indicates the directory of libraries
- **-l**: indicates the library name, the compiler can give the "**lib**" prefix to the library name and follows with **.so** as extension name.

BY NC SA

Using shared library

- After the “main” has been compiled, try to run it:

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ ls
libmymath.so  main  main.cpp  mymath.cpp  mymath.h
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ ./main
./main: error while loading shared libraries: libmymath.so: cannot open shared object file: No such file or directory
```

- It failed because “main” now relys on “libmymath.so”. By default, libraries are located in **/usr/local/lib** or **/usr/lib** , but our “libmymath.so” is not in that directory. You must tell the terminal where to find “libmymath.so”.

BY NC SA

Using a shared library

- Using **export** command to set environment variable "**LD_LIBRARY_PATH**"
- And then run "main" again

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ echo $LD_LIBRARY_PATH
.:
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ ./main
NULL pointer!
The result1 is 36
The result2 is 0
```

export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH

There is no space on either side of the equal sign

. indicates the current directory

Another choice is to move your .so file to **/usr/lib** folder by **mv** command

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ sudo mv libmymath.so /usr/lib
[sudo] password for maydlee:
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ ./main
NULL pointer!
The result1 is 36
The result2 is 0
```

BY NC SA

Shared library in makefile

SHAREDLIB [WSL: UBUNTU]
- main.cpp
- Makefile
- mymath.cpp
- mymath.h

All the files are in the same folder.

```makefile
# makefile with dynamic library

.PHONY: libd testlibd clean

libd: libfunction.so
libfunction.so: mymath.cpp
	g++ -shared -fPIC -o libfunction.so mymath.cpp

testlibd : main
main : main.cpp libfunction.so
	g++ main.cpp -L. -lfunction -Wl,-rpath=. -o main

clean:
	rm -rf *.o *.so main
```

–Wl option allows you to pass subsequent arguments to the linker.
–rpath option is used to specify the directories that the runtime dynamic linker(ld.so) should search when looking for dynamic libraries during execution.

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ make
g++ -shared -fPIC -o libfunction.so mymath.cpp
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ make testlibd
g++ main.cpp -L. -lfunction -Wl,-rpath=. -o main
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ ./main
NULL pointer!
The result1 is 36
The result2 is 0
```

Run the first target to create the dynamic library.
Run the second target to link the dynamic library to the executable file.
Run the executable file.

BY NC SA

```
SHAREDLIB [WSL: UBU...
  include
    mymath.h
  src
    main.cpp
    mymath.cpp
  Makefile
```

This time we put all the source files in the “src” folder, the function header file in the “include” folder, and create a makefile in the current folder.

```makefile
cpp_srcs := $(wildcard src/*.cpp)
cpp_objs := $(patsubst src/%.cpp, objs/%.o, $(cpp_srcs))
so_objs  := $(filter-out objs/main.o, $(cpp_objs))

include_path := ./include

I_options := $(include_path:%=-I%)

compile_flags := -g -O3 -w -fPIC $(I_options)

objs/%.o : src/%.cpp
    mkdir -p $(dir $@)
    g++ -c $^ -o $@ $(compile_flags)

compile : $(cpp_objs)

# ============== Generating dynamic library ==============
lib/libfunction.so : $(so_objs)
    mkdir -p $(dir $@)
    g++ -shared $^ -o $@

dynamic : lib/libfunction.so

clean:
    rm -rf ./objs ./lib
.PHONY : compile dynamic clean
```

The first part of the makefile just creates a dynamic library named **libfunction.so**

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ make
mkdir -p objs/
g++ -c src/mymath.cpp -o objs/mymath.o -g -O3 -w -fPIC -I./include
mkdir -p objs/
g++ -c src/main.cpp -o objs/main.o -g -O3 -w -fPIC -I./include
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ make dynamic
mkdir -p lib/
g++ -shared objs/mymath.o -o lib/libfunction.so
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ cd lib
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib/lib$ ls
libfunction.so
```

CC BY NC SA

```makefile
# ============== linking dynamic library ==============
library_path := ./lib

linking_libs := function

l_options := $(linking_libs:%=-l%)
L_options := $(library_path:%=-L%)
r_options := $(library_path:%=-Wl,-rpath=%)

objs/testdynamic : objs/main.o compile dynamic
	mkdir -p $(dir $@)
	g++ $< -o $@ $(linking_flags)

run : objs/testdynamic
	./$<

clean :
	rm -rf lib objs

.PHONY : compile dynamic run clean
```

The second part of the makefile links the dynamic library **libfunction.so** to the executable file **testdynamic** in the "objs" folder.

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/mycode/CcodeVS/sharedlib$ make run
mkdir -p objs/
g++ objs/main.o -o objs/testdynamic -lfunction -L./lib -Wl,-rpath=./lib
./objs/testdynamic
NULL pointer!
The result1 is 36
The result2 is 0
```

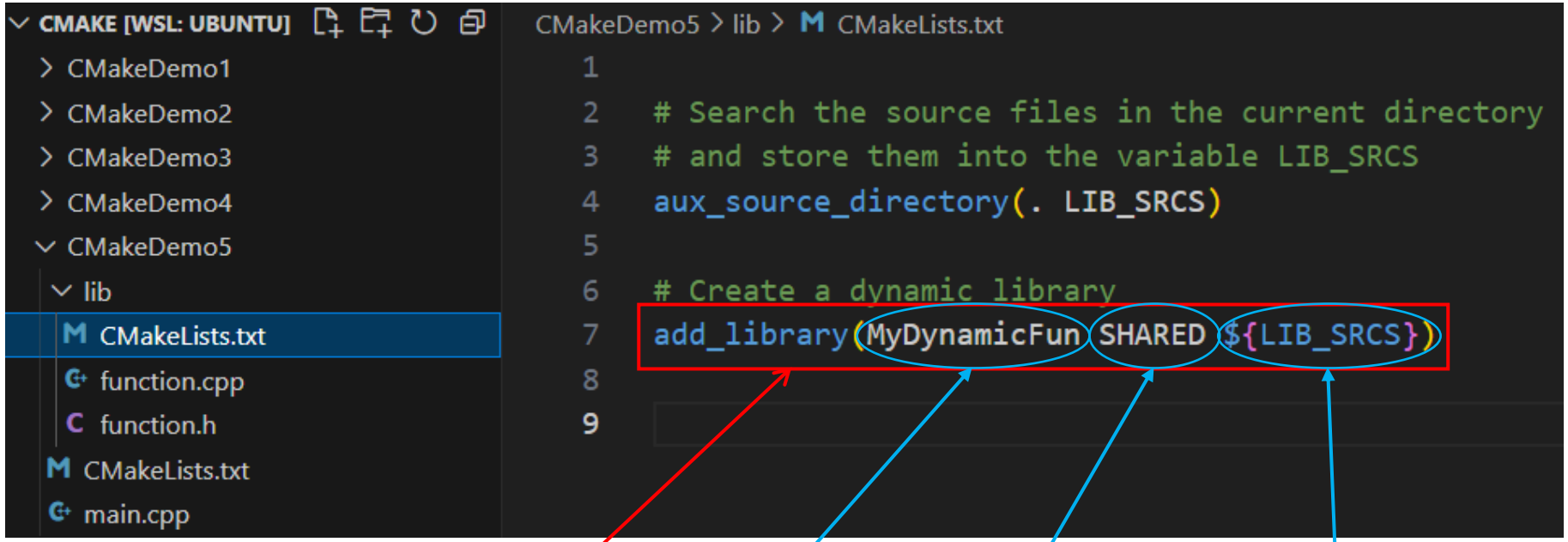
BY NC SA

Creating and linking a dynamic library by CMake

We want to create a dynamic library by function.cpp and call the dynamic library in main.cpp. This time we write two CMakeLists.txt files, one in **CmakeDemo5** folder and another in **lib** folder.

```
./CMakeDemo5
    |
    +--- main.cpp
    |
    +--- lib/
        |
        +--- function.h
        |
        +--- function.cpp
```

The CMakeLists.txt in **lib** folder creates a dynamic library.



library file name

dynamic library

The directory from which the library file originates.

Create a static library named libMyDynamicFun.so by the files in the current directory.

BY NC SA

The CMakeLists.txt in **CMakeDemo5** folder creates the project.

```
CMAKE [WSL: UBUNTU]
> CMakeDemo1
> CMakeDemo2
> CMakeDemo3
> CMakeDemo4
v CMakeDemo5
  v lib
    M CMakeLists.txt
    function.cpp
    function.h
  M CMakeLists.txt
  main.cpp
M CMakeLists.txt
hello.cpp
M Makefile
```

CMakeDemo5 > M CMakeLists.txt

```
# CMake minimum version
cmake_minimum_required(VERSION 3.10)

# project information
project(CMakeDemo5)

# Search the source files in the current directory
# and store them into the variable DIR_SRCS
aux_source_directory(. DIR_SRCS)

# add the directory of include
include_directories(lib)

# add the subdirectory of lib
add_subdirectory(lib)

# Specify the build target
add_executable(CMakeDemo5 ${DIR_SRCS})

# Add the dynamic library
target_link_libraries(CMakeDemo5 MyDynamicFun)
```

add_subdirectory command indicates there is a subdirectory in the project. When running the command, it will execute the CMakeLists.txt in the subdirectory automatically.

Indicates that the project needs link a library named **MyDynamicFun**, MyDynamicFun can be a static library file or a dynamic library file.

project name

library file name
If there are more than one file, list them using space as the separator.

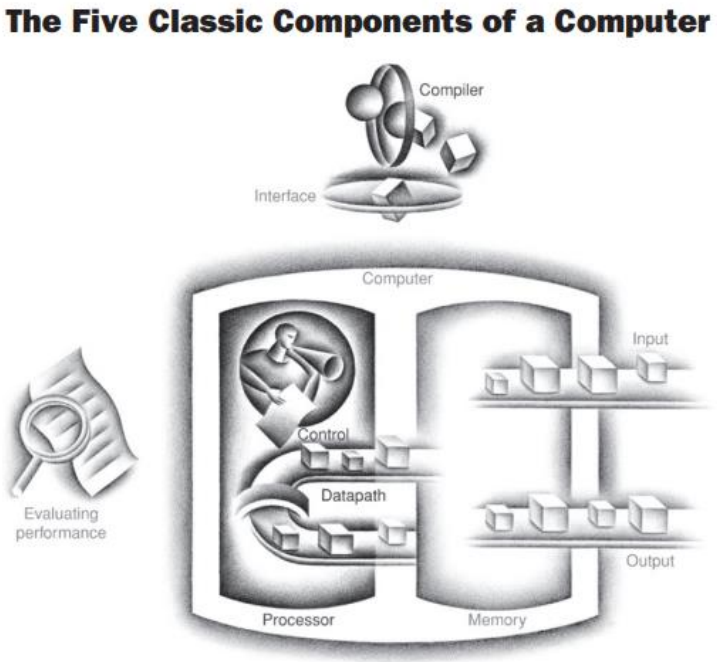
BY NC SA

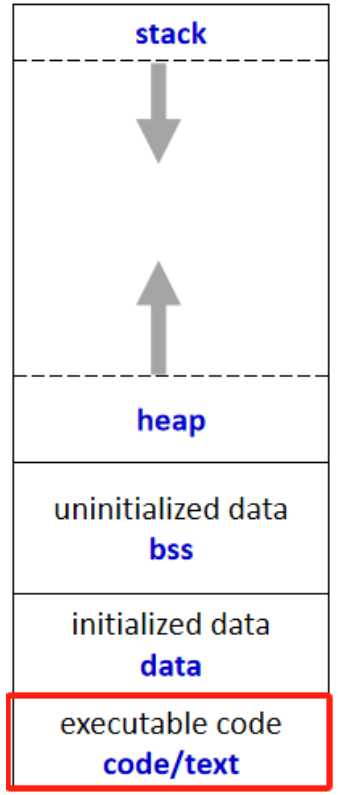
```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/CMake/CMakeDemo5$ mkdir build
maydlee@LAPTOP-U1MO0N2F:/mnt/d/CMake/CMakeDemo5$ cd build
maydlee@LAPTOP-U1MO0N2F:/mnt/d/CMake/CMakeDemo5/build$ cmake ..
-- The C compiler identification is GNU 9.4.0
-- The CXX compiler identification is GNU 9.4.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /mnt/d/CMake/CMakeDemo5/build
```

```
maydlee@LAPTOP-U1MO0N2F:/mnt/d/CMake/CMakeDemo5/build$ make
Scanning dependencies of target MyDynamicFun
[ 25%] Building CXX object lib/CMakeFiles/MyDynamicFun.dir/function.cpp.o
[ 50%] Linking CXX shared library libMyDynamicFun.so
[ 50%] Built target MyDynamicFun
Scanning dependencies of target CMakeDemo5
[ 75%] Building CXX object CMakeFiles/CMakeDemo5.dir/main.cpp.o
[100%] Linking CXX executable CMakeDemo5
[100%] Built target CMakeDemo5
```

BY NC SA

Function and Memory(1)





lab7 > C test_function.c > main()

```c
int add(int a, int b){
    return a+b;
}

int main(){
    int x=0,y=0,z=0;
    int (*func_pt)(int,int)=add;
    printf("address of data x:%p, y:%p, z=%p\n",&x,&y,&z);
    printf("address of func \"main\": %p\n",main);
    printf("address of func \"printf\": %p\n",printf);
    x=add(1,2);
    printf("address of func \"add\": %p\n",add);
    y=add(10,20);
    printf("address of func \"add\": %p\n",add);
    z=func_pt(100,200);
    printf("value of func pointer \"func_pt\": %p\n",func_pt);
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$ gcc test_function.c
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$ ./a.out
address of data x:0x7ffef08e9ca4, y:0x7ffef08e9ca8, z=0x7ffef08e9cac
address of func "main": 0x558569f07171
address of func "printf": 0x7fdee8818c90
address of func "add": 0x558569f07159
address of func "add": 0x558569f07159
value of func pointer "func_pt": 0x558569f07159
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$
```

CC BY NC SA

Function and Memory(2)

Q. What the specific memory usage during the execution of the “p3_demo1” program ?

lab7 > mymath.h > ...

```c
#ifndef __MY_MATH_H__
#define __MY_MATH_H__

int add(int a, int b);
#endif
```

lab7 > mymath.c > add(int, int)

```c
#include "mymath.h"

int add(int a, int b){
    return a+b;
}
```

lab7 > p2_demo1.c > ...

```c
#include <stdio.h>
#include "mymath.h"
int main(){
    int x=10,y=10;
    scanf("%d",&x);
    scanf("%d",&y);
    int sum=add(x,y);
    return 0;
}
```

```
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$ gcc -shared -fPIC -o libmymath.so mymath.c
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$ gcc -o p2_demo1 p2_demo1.c -L. -lmymath
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$ ./p2_demo1
./p2_demo1: error while loading shared libraries: libmymath.so: cannot open shared object file: No such file or directory
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$ export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$ ./p2_demo1
```

Tips:

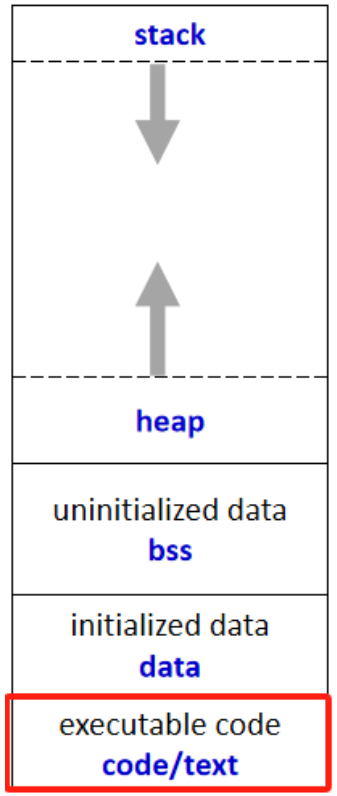
1. Use “pgrep” tool to view the process number of the currently running process that matches the specified name
2. Use “cat” to find current usage of memory by the process according to the “maps” of the process.

The “perms” field is a set of permissions: r = read, w = write, x = execute, s = shared, p = private (copy on write)

https://man7.org/linux/man-pages/man5/proc_pid_maps.5.html

BY NC SA

Function and Memory(3)




```
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$ pgrep p2_demo1
85257
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$ cat /proc/85257/maps
559197e68000-559197e69000 r--p 00000000 00:53 8444249301347143          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/p2_demo1
559197e69000-559197e6a000 r-xp 00001000 00:53 8444249301347143          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/p2_demo1
559197e6a000-559197e6b000 r--p 00002000 00:53 8444249301347143          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/p2_demo1
559197e6b000-559197e6c000 r--p 00002000 00:53 8444249301347143          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/p2_demo1
559197e6c000-559197e6d000 rw-p 00003000 00:53 8444249301347143          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/p2_demo1
5591ac12d000-5591ac14e000 rw-p 00000000 00:00 0                          [heap]
7fdd9f653000-7fdd9f656000 rw-p 00000000 00:00 0
7fdd9f656000-7fdd9f678000 r--p 00000000 08:20 43481                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fdd9f678000-7fdd9f7f0000 r-xp 00022000 08:20 43481                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fdd9f7f0000-7fdd9f83e000 r--p 0019a000 08:20 43481                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fdd9f83e000-7fdd9f842000 r--p 001e7000 08:20 43481                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fdd9f842000-7fdd9f844000 rw-p 001eb000 08:20 43481                      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fdd9f844000-7fdd9f848000 rw-p 00000000 00:00 0
7fdd9f851000-7fdd9f852000 r--p 00000000 00:53 7599824371214859          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/libmymath.so
7fdd9f852000-7fdd9f853000 r-xp 00001000 00:53 7599824371214859          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/libmymath.so
7fdd9f853000-7fdd9f854000 r--p 00002000 00:53 7599824371214859          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/libmymath.so
7fdd9f854000-7fdd9f855000 r--p 00002000 00:53 7599824371214859          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/libmymath.so
7fdd9f855000-7fdd9f856000 rw-p 00003000 00:53 7599824371214859          /mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7/libmymath.so
7fdd9f856000-7fdd9f858000 rw-p 00000000 00:00 0
7fdd9f858000-7fdd9f859000 r--p 00000000 08:20 43477                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fdd9f859000-7fdd9f87c000 r-xp 00001000 08:20 43477                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fdd9f87c000-7fdd9f884000 r--p 00024000 08:20 43477                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fdd9f885000-7fdd9f886000 r--p 0002c000 08:20 43477                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fdd9f886000-7fdd9f887000 rw-p 0002d000 08:20 43477                      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fdd9f887000-7fdd9f888000 rw-p 00000000 00:00 0
7ffca6082000-7ffca60a3000 rw-p 00000000 00:00 0                          [stack]
7ffca6162000-7ffca6166000 r--p 00000000 00:00 0                          [vvar]
7ffca6166000-7ffca6168000 r-xp 00000000 00:00 0                          [vdso]
ww2@DESKTOP-4NIH4UK:/mnt/c/Users/sustech/Desktop/C_CPP_CODE/lab7$
```

CC BY NC SA

Exercise 1

1. Overload a function **bool vabs(int * p, int n)** which can compute the absolute value for every element of an array, the array can be int, float and double.
   - Should n be int or size_t? what's the difference? Choose the better one.
   - Remember to check whether the pointer is valid.
2. Create a shared library “libvabs.so” with the 3 overloaded vabs() functions, and then link the shared library to generate your own program. Print the addresses of all vabs() functions.
3. Create a static library “libvabs.a” with the 3 overloaded vabs() functions, and then link the shared library to generate your own program. Print the addresses of all vabs() functions.

Expliain the differences between static library and shared library according to your experimental results.

BY NC SA

Exercise 2

```
struct stuinfo{
  string name;
   int age;
};
```

The prototype of the Compare:

```
template <typename T>
int Compare(const T &a, const T &b);
```

The output:

```
Compare of the two integers:-1
Compare of the two floats:1
Compare of the two characters:1
Compare of the two structs:1
```

Create a function template named Compare. This function should take two arguments of the same type. It should compare their values and return an integer based on the result:

Return 1 if the first argument is greater than the second.

Return -1 if the first argument is less than the second.

Return 0 if both values are equal.

Test your function by calling it with different data types and printing the results. You must test with: two integers, two floating-point numbers, and two characters (char)

Now, you have a stuinfo structure that stores a student‘s name(string) and age(integer). Create an explicit specialization of your Compare template for this stuinfo structure. In this specialized version, the comparison should be based only on the student's age member.

Test this specialized function and print the result of comparing two stuinfo objects.

BY NC SA